

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

федеральное государственное автономное образовательное
учреждение высшего образования



**«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»**

Инженерная школа информационных технологий и робототехники

09.04.04 «Программная инженерия»

Отделение информационных технологий

Отчет по лабораторной работе №2

по дисциплине «Численные методы решения прикладных задач»

Выполнил:

Студент группы 8МП5Л

_____ Шаврин А.П.

Проверил:

К.х.н., доцент (ОНД, ИШПР)

_____ Зарубин А.Г.

Цель

Изучить и проработать базовое решение (baseline) задачи прогнозирования количества двух продуктов нефтехимического производства, предложенное организаторами хакатона SIBUR Challenge 2023. Освоить построение регуляризованных линейных моделей (Ridge-регрессия) и методы feature engineering для повышения качества прогноза и устойчивости модели на данных, выходящих за границы обучающей выборки (задача генерализации).

Задание

1. Ознакомиться с условием задачи №2 хакатона SIBUR Challenge 2023 «Генерализация».
2. Загрузить датасет train.parquet.
3. Воспроизвести baseline-решение организаторов на основе CatBoostRegressor.
4. Зафиксировать метрики качества (MAPE) baseline.
5. Предложить более точное решение на основе Ridge-регрессии с расширением признакового пространства.
6. Сравнить полученные результаты с baseline и сделать выводы.

Выполнение работы

1. Описание baseline-решения

Сначала была выполнена настройка окружение и импорт библиотек. Подключаются библиотеки для визуализации, работы с массивами и таблицами, разделения данных, расчета метрики и построения модели CatBoost. Устанавливается стиль графиков. Инициализируется директория данных и случайное состояние. Код приведен на рисунке 1.

```
1  from matplotlib import pyplot as plt
2
3  plt.style.use("bmh")
4  plt.rcParams["axes.titlesize"] = "large"
5
6  import pathlib
7  import numpy as np
8  import pandas as pd
9  pd.set_option('display.max_columns', None)
10
11  from sklearn.model_selection import train_test_split
12  from sklearn.metrics import mean_absolute_percentage_error
13  from catboost import CatBoostRegressor
14
15  # Пути и seed
16  DATA_DIR = pathlib.Path("./")
17  RS = 42
18
```

Рисунок 1. Настройка окружения

После чего произведена загрузка данных и их визуализация следующим блоком кода представленным на рисунке 2, вывод загруженного датасета представлен на рисунке 3 а гистограммы таргетов на рисунке 4. Оба таргета лежат в диапазоне [0, 100], распределение скошено влево — большинство значений малые

```
18
19  # Загрузка данных
20  train = pd.read_parquet(DATA_DIR.joinpath("train.parquet"))
21  print(train.head())
22
23  # Признаки и таргеты
24  CAT = ["feature4"]
25  TARGETS = ["target0", "target1"]
26  FTS = train.filter(like="feature").columns.difference(CAT)
27
28  # Гистограмма таргетов
29  plt.figure(figsize=(6, 6))
30  train[TARGETS].plot(kind="hist", range=(0, 100), bins=20, alpha=0.6, ax=plt.gca())
31  plt.xlabel("target")
32  plt.show()
33
```

Рисунок 2. Загрузка данных и их визуализация.

```
uv run python baseline.py
```

	feature0	feature1	feature2	feature3	feature4	feature5	feature6	
0	32.910908	17.376350	77.557840	2.929855	gas1	20.487150	57.633085	
1	41.263782	22.419445	47.945514	-25.847472	gas2	21.461239	3.474080	
2	25.580283	17.376350	77.654180	2.799411	gas1	20.487150	28.938295	
3	33.756900	17.376350	73.049625	2.953982	gas1	20.487150	28.932311	
4	4.223732	38.772534	48.015553	-25.843943	gas2	24.635721	12.011581	

	feature7	feature8	feature9	feature10	feature11	feature12	
0	49.245392	-44.124381	23.400064	-28.640648	-10.995420	40.856313	
1	49.659980	-13.553188	-13.047593	-12.445940	16.657368	42.453280	
2	49.245392	-44.124381	20.610679	-28.641150	-1.886830	40.927941	
3	49.245392	-44.124381	18.107963	-28.642094	16.284415	40.869406	
4	51.030938	84.244199	-17.735680	40.395582	16.557978	65.988300	

	feature13	feature14	feature15	feature16	feature17	feature18	
0	29.912800	33.168114	18.769718	41.812507	14.203652	9.689942	
1	99.792375	127.159321	-13.180244	50.722470	-14.451904	-0.906120	
2	16.340674	60.760647	18.658501	52.731646	14.395054	9.779019	
3	43.707663	48.037510	18.771030	52.305692	14.184314	9.798969	
4	91.081310	150.949700	-13.172169	51.012294	-14.439953	8.262354	

	feature19	feature20	feature21	feature22	feature23	feature24	
0	17.951627	24.198589	102.448710	110.529868	56.817260	12.887802	
1	105.724742	-9.435222	-16.060150	37.286110	61.224272	39.822424	
2	17.951638	24.999453	101.728213	111.471534	52.664304	12.887802	
3	17.951675	23.990300	101.312113	115.589451	56.840719	12.887802	
4	90.187207	-2.901661	-16.060150	37.398779	40.488468	128.295838	

	target0	target1
0	27.050891	6.502743
1	84.127890	76.578716
2	22.080133	3.036043
3	30.234082	8.910795
4	71.128092	50.475082

Рисунок 3. Вывод загруженного датасета.

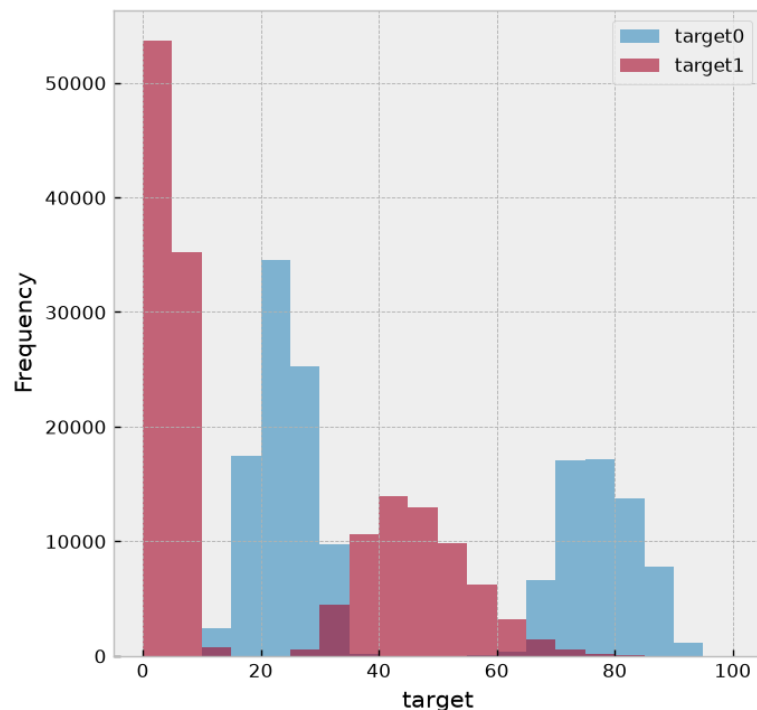


Рисунок 4. Гистограммы таргетов

Затем была выполнена предобработка данных, заключающаяся в кодирование категориального признака feature4 бинарно. Gas1 -> 0, gas2 -> 1.

Новый столбец `gas` добавляется к списку признаков. Итоговое число признаков — 25. Также выборка была разделена на тренировочную и валидационную. Код представлен на рисунке 5.

```
34 # Кодирование категориального признака
35 train["gas"] = 0
36 train.loc[train.feature4 == "gas2", "gas"] = 1
37
38 FTS = FTS.union(["gas"])
39
40 # Разделение на train и valid
41 X_tr, X_val, y_tr, y_val = train_test_split(
42     train[FTS], train[TARGETS], train_size=0.5, random_state=RS
43 )
44
```

Рисунок 5. График модели.

После чего для каждого таргета обучается отдельная модель CatBoost:

`max_depth` ограничено 4 для борьбы с переобучением, за счет не глубоких деревьев, максимум итераций бустинга установлен на 5000, остановка при отсутствии улучшений производится после 20 итераций, оптимизируется целевая метрика MAPE. Код на рисунке 6, а результаты обучения на рисунке 7 и 8.

```
45 # Обучение отдельной модели для каждого таргета
46 models = {}
47 tr_preds = []
48 val_preds = []
49
50 for tg in TARGETS:
51     print(f"{tg}", "=" * 10)
52     cb_model = CatBoostRegressor(
53         max_depth=4,
54         iterations=5000,
55         early_stopping_rounds=20,
56         objective="MAPE",
57         verbose=200,
58         random_state=RS
59     )
60     cb_model.fit(X_tr, y_tr[tg], eval_set=(X_val, y_val[tg]))
61
62     tr_preds.append(cb_model.predict(X_tr))
63     val_preds.append(cb_model.predict(X_val))
64
65     models[tg] = cb_model
```

Рисунок 6. Обучение моделей.

target0 =====					
0:	learn: 0.3849499	test: 0.3882364	best: 0.3882364 (0)	total: 55.2ms	remaining: 4m 35s
200:	learn: 0.0651563	test: 0.0653452	best: 0.0653452 (200)	total: 554ms	remaining: 13.2s
400:	learn: 0.0238987	test: 0.0238584	best: 0.0238584 (400)	total: 1.09s	remaining: 12.5s
600:	learn: 0.0147546	test: 0.0148271	best: 0.0148271 (600)	total: 1.59s	remaining: 11.7s
800:	learn: 0.0120293	test: 0.0121300	best: 0.0121300 (800)	total: 2.12s	remaining: 11.1s
1000:	learn: 0.0103850	test: 0.0104976	best: 0.0104976 (1000)	total: 2.67s	remaining: 10.7s
1200:	learn: 0.0093929	test: 0.0095225	best: 0.0095225 (1200)	total: 3.23s	remaining: 10.2s
1400:	learn: 0.0086623	test: 0.0088086	best: 0.0088086 (1400)	total: 3.77s	remaining: 9.69s
1600:	learn: 0.0080397	test: 0.0082001	best: 0.0082001 (1600)	total: 4.32s	remaining: 9.18s
1800:	learn: 0.0075825	test: 0.0077523	best: 0.0077523 (1800)	total: 4.87s	remaining: 8.64s
2000:	learn: 0.0072116	test: 0.0073942	best: 0.0073942 (2000)	total: 5.41s	remaining: 8.11s
2200:	learn: 0.0069329	test: 0.0071276	best: 0.0071276 (2200)	total: 5.95s	remaining: 7.56s
2400:	learn: 0.0066694	test: 0.0068721	best: 0.0068721 (2400)	total: 6.48s	remaining: 7.02s
2600:	learn: 0.0064451	test: 0.0066548	best: 0.0066548 (2600)	total: 7.02s	remaining: 6.47s
2800:	learn: 0.0062253	test: 0.0064391	best: 0.0064391 (2800)	total: 7.55s	remaining: 5.93s
3000:	learn: 0.0060239	test: 0.0062437	best: 0.0062437 (3000)	total: 8.1s	remaining: 5.39s
3200:	learn: 0.0058242	test: 0.0060462	best: 0.0060462 (3200)	total: 8.66s	remaining: 4.87s
3400:	learn: 0.0056385	test: 0.0058639	best: 0.0058639 (3400)	total: 9.22s	remaining: 4.33s
3600:	learn: 0.0054961	test: 0.0057249	best: 0.0057249 (3600)	total: 9.77s	remaining: 3.79s
3800:	learn: 0.0053721	test: 0.0056060	best: 0.0056060 (3800)	total: 10.3s	remaining: 3.25s
4000:	learn: 0.0052362	test: 0.0054708	best: 0.0054708 (4000)	total: 10.9s	remaining: 2.71s
4200:	learn: 0.0051247	test: 0.0053643	best: 0.0053643 (4200)	total: 11.4s	remaining: 2.17s
4400:	learn: 0.0050011	test: 0.0052447	best: 0.0052447 (4400)	total: 12s	remaining: 1.63s
4600:	learn: 0.0048960	test: 0.0051432	best: 0.0051432 (4600)	total: 12.5s	remaining: 1.09s
4800:	learn: 0.0047911	test: 0.0050415	best: 0.0050415 (4800)	total: 13.1s	remaining: 543ms
4999:	learn: 0.0047023	test: 0.0049551	best: 0.0049551 (4999)	total: 13.6s	remaining: 0us
bestTest = 0.004955145626					
bestIteration = 4999					

Рисунок 7. Обучение модели первого таргета.

target1 =====					
0:	learn: 0.6367980	test: 0.6397262	best: 0.6397262 (0)	total: 4.32ms	remaining: 21.6s
200:	learn: 0.2847067	test: 0.2911762	best: 0.2911762 (200)	total: 526ms	remaining: 12.6s
400:	learn: 0.1309718	test: 0.1344560	best: 0.1344560 (400)	total: 1.06s	remaining: 12.2s
600:	learn: 0.0965694	test: 0.0993640	best: 0.0993640 (600)	total: 1.62s	remaining: 11.8s
800:	learn: 0.0830974	test: 0.0857186	best: 0.0857186 (800)	total: 2.18s	remaining: 11.4s
1000:	learn: 0.0760966	test: 0.0785989	best: 0.0785989 (1000)	total: 2.74s	remaining: 10.9s
1200:	learn: 0.0712650	test: 0.0737149	best: 0.0737149 (1200)	total: 3.29s	remaining: 10.4s
1400:	learn: 0.0679034	test: 0.0703281	best: 0.0703281 (1400)	total: 3.83s	remaining: 9.85s
1600:	learn: 0.0657070	test: 0.0681096	best: 0.0681096 (1600)	total: 4.38s	remaining: 9.3s
1800:	learn: 0.0595853	test: 0.0619795	best: 0.0619795 (1800)	total: 4.92s	remaining: 8.73s
2000:	learn: 0.0548336	test: 0.0572411	best: 0.0572411 (2000)	total: 5.46s	remaining: 8.19s
2200:	learn: 0.0527635	test: 0.0551645	best: 0.0551645 (2200)	total: 6.03s	remaining: 7.67s
2400:	learn: 0.0487392	test: 0.0510544	best: 0.0510544 (2400)	total: 6.59s	remaining: 7.13s
2600:	learn: 0.0458422	test: 0.0479959	best: 0.0479959 (2600)	total: 7.14s	remaining: 6.59s
2800:	learn: 0.0426919	test: 0.0446183	best: 0.0446183 (2800)	total: 7.71s	remaining: 6.05s
3000:	learn: 0.0402616	test: 0.0420781	best: 0.0420781 (3000)	total: 8.27s	remaining: 5.51s
3200:	learn: 0.0381582	test: 0.0398457	best: 0.0398447 (3199)	total: 8.84s	remaining: 4.97s
3400:	learn: 0.0360326	test: 0.0376691	best: 0.0376691 (3400)	total: 9.39s	remaining: 4.42s
3600:	learn: 0.0352491	test: 0.0368364	best: 0.0368352 (3599)	total: 9.96s	remaining: 3.87s
3800:	learn: 0.0326455	test: 0.0339206	best: 0.0339196 (3796)	total: 10.5s	remaining: 3.32s
4000:	learn: 0.0317444	test: 0.0329683	best: 0.0329683 (4000)	total: 11.1s	remaining: 2.77s
4200:	learn: 0.0304229	test: 0.0315987	best: 0.0315987 (4200)	total: 11.6s	remaining: 2.21s
4400:	learn: 0.0293846	test: 0.0305186	best: 0.0305186 (4400)	total: 12.2s	remaining: 1.66s
4600:	learn: 0.0278145	test: 0.0288905	best: 0.0288905 (4600)	total: 12.8s	remaining: 1.11s
4800:	learn: 0.0270882	test: 0.0281453	best: 0.0281453 (4800)	total: 13.3s	remaining: 552ms
4999:	learn: 0.0266403	test: 0.0277042	best: 0.0277041 (4997)	total: 13.9s	remaining: 0us
bestTest = 0.02770406146					
bestIteration = 4997					

Рисунок 8. Обучение модели второго таргета.

После чего предсказания двух моделей объединяются в матрицу ($n \times 2$). Значения обрезаются в диапазон $[0, 100]$, так как таргеты — это количества продукта и не могут быть отрицательными или превышать 100. А затем на

результате обучения 2х моделей производится расчет метрик. Код представлен на рисунке 9, а результат расчета метрик на рисунке 10.

```
67 # Сборка предсказаний и обрезка по диапазону таргетов
68 tr_preds = np.column_stack(tr_preds)
69 val_preds = np.column_stack(val_preds)
70
71 tr_preds = np.clip(tr_preds, 0, 100)
72 val_preds = np.clip(val_preds, 0, 100)
73
74 # Метрики
75 print(f"MAPE (train): {mean_absolute_percentage_error(y_tr, tr_preds) * 100:.3f} %")
76 print(f"MAPE (val): {mean_absolute_percentage_error(y_val, val_preds) * 100:.3f} %")
77
```

Рисунок 9. Код сбора предсказаний и расчета метрик.

```
Shrink model to first 4998 iterations.
MAPE (train): 1.567 %
MAPE (val): 1.633 %
```

Рисунок 10. Результат расчета метрик.

2. Вывод по baseline

Baseline-решение организаторов представляет собой классический подход с использованием градиентного бустинга (CatBoost) и демонстрирует следующие характеристики:

Достоинства:

- Простота реализации и интерпретации.
- Отсутствие необходимости в сложной предобработке данных (пропусков нет, категориальный признак всего один).
- Высокое качество на локальной валидации: MAPE = 1.633%, что соответствует уровню, близкому к топовым решениям.

Недостатки и ключевая проблема:

- Расхождение локальной валидации с лидербордом. Автор baseline в README прямо отмечает: модели, дающие ~1.6% MAPE на локальной валидации, на публичном лидерборде показывают около 10%. Это означает, что модель переобучается под train/valid-распределение.
- Плохая генерализация. CatBoost — модель на основе решающих деревьев. Деревья не могут экстраполировать: предсказание всегда лежит

внутри диапазона значений, виденных в обучающей выборке. Для задачи «цифровых экспериментов» (проверка нестандартных режимов) это критично.

- Игнорирование структуры данных. Не учитывается влияние типа газа (для gas1 и gas2 можно строить отдельные модели), не используются нелинейные преобразования признаков.

3. Проверка гипотезы о нелинейности данных

Baseline-решение на основе CatBoost показывает высокое качество на локальной валидации (MAPE = 1.633%), однако оно не удовлетворяет главному требованию задачи — генерализации. Деревья решений не экстраполируют: предсказание всегда лежит внутри диапазона значений таргета, наблюдавшихся в обучающей выборке. Именно этим объясняется расхождение локальной валидации (~1.6%) с публичным лидербордом (~10%).

Для решения этой проблемы предлагается заменить CatBoost на Ridge-регрессию — линейную модель с L2-регуляризацией. Линейные модели экстраполируют за пределы обучающей выборки плавно и предсказуемо. Однако у Ridge есть своя слабость: модель линейна по построению и не улавливает нелинейные зависимости автоматически. Поэтому перед построением Ridge необходимо убедиться, что нелинейности в данных действительно есть, и определить, для каких признаков нужно добавить нелинейные преобразования.

Для проверки наличия нелинейных зависимостей была обучена обычная линейная регрессия (OLS) и построен график остатков — зависимость остатка $y - \hat{y}$ от предсказания $\hat{y}$. Идея метода: если линейная модель описывает данные адекватно, остатки должны представлять собой случайное облако вокруг нуля без выраженной структуры. Если же остатки имеют форму, это означает, что линейная модель не улавливает часть закономерности, то есть в данных присутствует нелинейность. Код построения графика остатков приведен на рисунке 11, результат — на рисунке 12.

```

41
42 # Residual plot для линейной модели
43 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
44
45 for ax, tg in zip(axes, TARGETS):
46     lin = LinearRegression().fit(X_tr, y_tr[tg])
47     residuals = y_tr[tg] - lin.predict(X_tr)
48     for g_val, color in zip([0, 1], ["blue", "orange"]):
49         mask = X_tr["gas"] == g_val
50         ax.scatter(lin.predict(X_tr)[mask], residuals[mask],
51                   alpha=0.1, s=1, c=color, label=f"gas{g_val+1}")
52     ax.axhline(0, color="red", linestyle="--")
53     ax.legend()
54     ax.set_xlabel("Предсказание линейной модели")
55     ax.set_ylabel("Остаток")
56     ax.set_title(f"Residual plot для {tg}")
57
58 plt.tight_layout()
59 plt.savefig("residuals.png", dpi=100, bbox_inches="tight")
60 plt.close()
61

```

Рисунок 11. Код построения графика остатков.

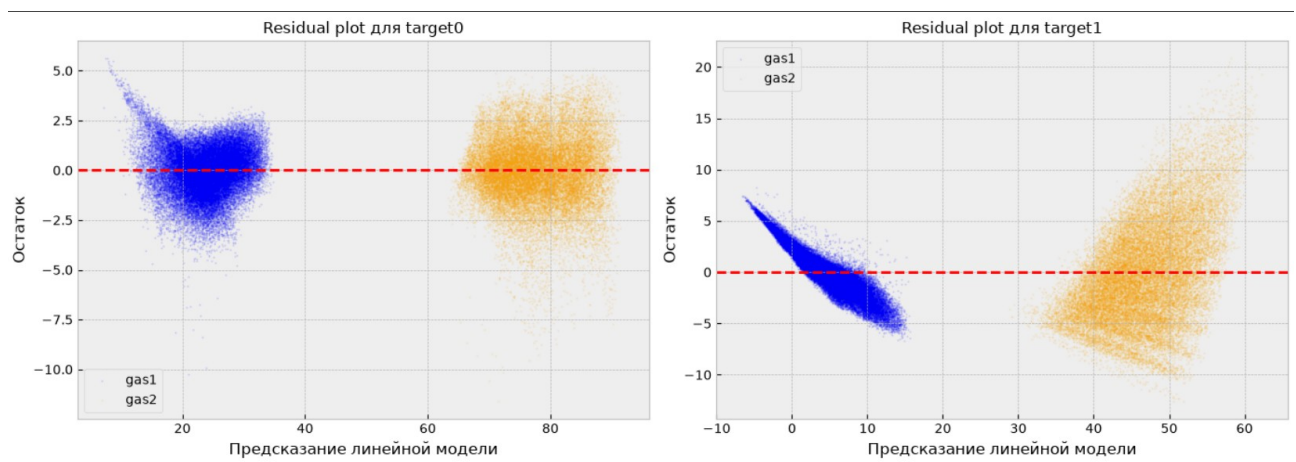


Рисунок 12. График остатков.

На графике остатков для обоих таргетов видны два отдельных кластера предсказаний с разрывом между ними. Внутри кластеров остатки образуют наклонные полосы и выраженные кривые. Это указывает на две проблемы одновременно: наличие структуры в остатках (признак нелинейности) и наличие двух режимов в данных (признак категориальной переменной). В данных есть единственный категориальный признак — feature4 со значениями gas1 и gas2 (тип газа). Для проверки residual plot был раскрашен по значению признака gas и как можно видеть на рисунке каждый режим окрашен своим цветом. Левый кластер для обоих таргетов целиком соответствует gas1 (синие точки), правый кластер — gas2 (оранжевые точки). Разрыв между кластерами совпадает с границей между типами газа. Это означает, что тип газа определяет

режим процесса: при gas1 таргеты принимают одни значения (около 25 и 5 соответственно), при gas2 — другие (около 75 и 45). Линейная модель, которая не различает два режима, вынуждена проводить одну общую прямую через оба кластера и систематически ошибается. Дополнительно внутри каждого кластера видна нелинейная структура: для target1 при gas1 остатки образуют выраженную кривую, для target0 в левом кластере — наклонную полосу. Это вторая нелинейность — уже внутри режима.

4. Анализ важности признаков

Раз в данных есть нелинейности, для Ridge необходимо добавить нелинейные преобразования признаков: квадраты, взаимодействия и степени. Однако если добавить их для всех 25 признаков, число новых признаков станет чрезмерным (только попарных взаимодействий — 600, а степеней — тысячи). Это замедлит обучение и может привести к переобучению.

Поэтому для нелинейных преобразований были выбраны только самые важные признаки. Для их определения использовалась важность признаков из CatBoost — модели, которая автоматически оценивает вклад каждого признака с учетом всех нелинейностей и взаимодействий. Код получения и визуализации важности приведен на рисунке 13, результат — на рисунке 14.

По результатам анализа наиболее важными признаками оказались:

- для target0: feature22 (62.1%), feature2 (27.4%), feature13 (4.3%), feature19 (1.9%), feature20 (1.5%);
- для target1: feature16 (49.0%), feature13 (6.3%), feature20 (5.4%), gas (5.1%), feature22 (4.7%).

Признак feature22 является доминирующим для target0, признак feature16 — для target1. Признак gas вошел в топ-10 только для target1, что указывает на его большее влияние именно на этот таргет. Эти признаки используются для добавления нелинейных преобразований в Ridge-регрессию.

```

64 # Обучение CatBoost (baseline организаторов)
65 models = {}
66 tr_preds = []
67 val_preds = []
68
69 for tg in TARGETS:
70     print(f"{tg}", "=" * 10)
71     cb_model = CatBoostRegressor(
72         max_depth=4,
73         iterations=5000,
74         early_stopping_rounds=20,
75         objective="MAPE",
76         verbose=200,
77         random_state=RS
78     )
79     cb_model.fit(X_tr, y_tr[tg], eval_set=(X_val, y_val[tg]))
80     tr_preds.append(cb_model.predict(X_tr))
81     val_preds.append(cb_model.predict(X_val))
82     models[tg] = cb_model
83
84 # Сборка и обрезка
85 tr_preds = np.clip(np.column_stack(tr_preds), 0, 100)
86 val_preds = np.clip(np.column_stack(val_preds), 0, 100)
87
88 # Метрики
89 print(f"MAPE (train): {mean_absolute_percentage_error(y_tr, tr_preds) * 100:.3f} %")
90 print(f"MAPE (val): {mean_absolute_percentage_error(y_val, val_preds) * 100:.3f} %")
91
92 # Важность признаков из CatBoost
93 importances = {}
94 for tg in TARGETS:
95     imp = models[tg].get_feature_importance()
96     importances[tg] = imp
97     print(f"\n== Топ-10 признаков для {tg} ==")
98     pairs = sorted(zip(FTS, imp), key=lambda x: -x[1])[:10]
99     for name, val in pairs:
100         print(f"{name}: {val:.2f}")
101
102 # Визуализация важности
103 fig, axes = plt.subplots(1, 2, figsize=(14, 6))
104 for ax, tg in zip(axes, TARGETS):
105     pairs = sorted(zip(FTS, importances[tg]), key=lambda x: x[1])
106     names = [p[0] for p in pairs]
107     vals = [p[1] for p in pairs]
108     ax.barh(names, vals)
109     ax.set_title(f"Важность признаков для {tg}")
110 plt.tight_layout()
111 plt.savefig("feature_importance.png", dpi=100, bbox_inches="tight")
112 plt.close()
113

```

Рисунок 15. Код получения важности признаков из CatBoost.

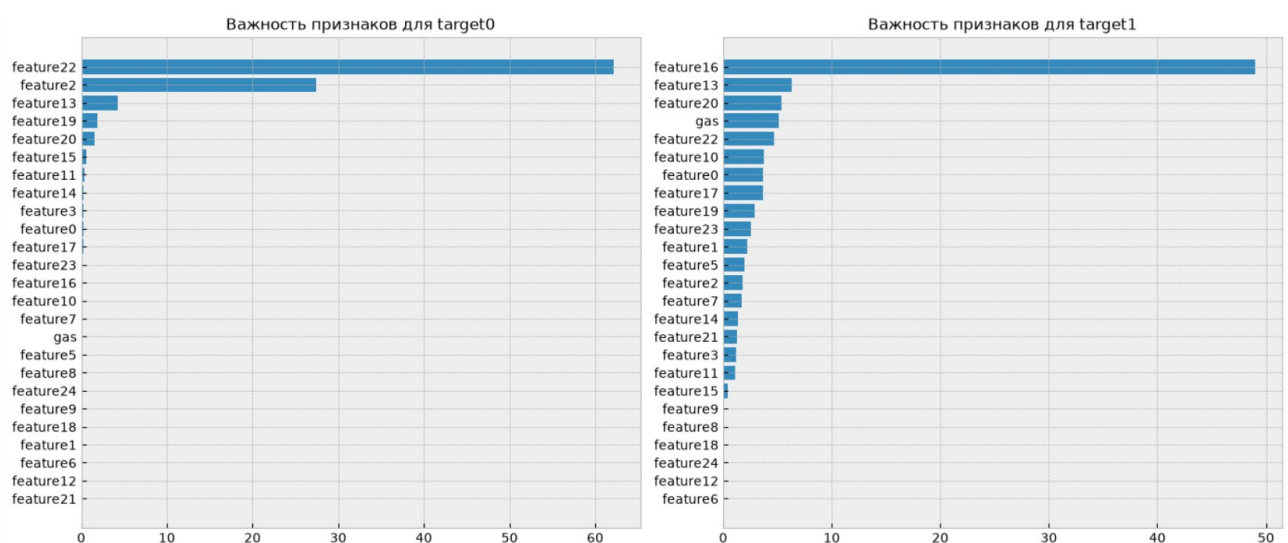


Рисунок 16. Важность признаков для target0 и target1.

5. Построение Ridge-регрессии

На основе результатов анализа нелинейности и важности признаков было принято решение заменить CatBoost на Ridge-регрессию с постепенным расширением признакового пространства. Для этого были построены пять вариантов моделей:

- OLS — обычная линейная регрессия на исходных 25 признаках. Служит отправной точкой для оценки вклада каждого последующего расширения.
- Ridge — линейная регрессия с L2-регуляризацией на исходных признаках. Перед обучением признаки масштабируются через StandardScaler — обязательное условие для корректной работы L2-регуляризации, поскольку штраф чувствителен к масштабу. Гиперпараметр λ подбирается автоматически через 5-кратную кросс-валидацию (RidgeCV).
- Ridge + квадраты — к исходным признакам добавлены их квадраты. Это дает модели возможность моделировать квадратичные зависимости — первое расширение, которое должно уловить нелинейности, выявленные на этапе анализа остатков.
- Ridge + взаимодействия — к квадратам добавлены произведения важных признаков (отобранных по важности из CatBoost) со всеми остальными. Такой отбор вместо всех 600 возможных попарных взаимодействий позволяет сохранить разумное число признаков и избежать переобучения.
- Ridge + степени 3–4 — финальное расширение: добавлены степени 3 и 4 для важных признаков. Это дает модели возможность моделировать более сложные нелинейные зависимости.

Код построения всех моделей приведен на рисунке 15, результаты обучения — на рисунке 16.

```

112 # Построение Ridge-регрессии с постепенным расширением признаков
113 idx_tr, idx_val = train_test_split(
114     np.arange(len(train)), train_size=0.5, random_state=RS
115 )
116 X_all = train[FTS].values
117 y_all = {tg: train[tg].values for tg in TARGETS}
118
119 def mape(y_true, y_pred):
120     return mean_absolute_percentage_error(y_true, y_pred) * 100
121
122 def fit_ridge(X_train, y_train, X_valid):
123     scaler = StandardScaler()
124     Xtr = scaler.fit_transform(X_train)
125     Xval = scaler.transform(X_valid)
126     model = RidgeCV(alphas=np.logspace(-3, 3, 20), cv=5)
127     model.fit(Xtr, y_train)
128     return model.predict(Xval)
129
130 # Важные признаки, отобранные по результатам CatBoost
131 important = {
132     "target0": ["feature22", "feature2", "feature13", "feature19", "feature20"],
133     "target1": ["feature16", "feature13", "feature20", "gas", "feature22"],
134 }
135
136 results = {"target0": {}, "target1": {}}
137 best_matrices = {} # будем хранить лучшую матрицу признаков для каждого таргета
138
139 for tg in TARGETS:
140     print(f'\n{'=' * 20} {tg} {'=' * 20}')
141     imp_idx = [list(FTS).index(f) for f in important[tg]]
142
143     # 1. OLS
144     lin = LinearRegression().fit(X_tr, y_tr[tg])
145     results[tg]["OLS"] = mape(y_val[tg], lin.predict(X_val))
146     print(f"OLS: {results[tg]['OLS']:.3f}%")
147
148     # 2. Ridge на исходных
149     pred = fit_ridge(X_tr, y_tr[tg], X_val)
150     results[tg]["Ridge"] = mape(y_val[tg], pred)
151     print(f"Ridge: {results[tg]['Ridge']:.3f}%")
152
153     # 3. Ridge + квадраты
154     X_sq = np.hstack([X_all, X_all ** 2])
155     pred = fit_ridge(X_sq[idx_tr], y_all[tg][idx_tr], X_sq[idx_val])
156     results[tg]["Ridge + квадраты"] = mape(y_all[tg][idx_val], pred)
157     print(f"Ridge + квадраты: {results[tg]['Ridge + квадраты']:.3f}%")
158
159     # 4. Ridge + взаимодействия
160     X_int = X_sq.copy()
161     for fi in imp_idx:
162         for j in range(X_all.shape[1]):
163             if j != fi:
164                 X_int = np.hstack([X_int, (X_all[:, fi] * X_all[:, j]).reshape(-1, 1)])
165     pred = fit_ridge(X_int[idx_tr], y_all[tg][idx_tr], X_int[idx_val])
166     results[tg]["Ridge + взаимодействия"] = mape(y_all[tg][idx_val], pred)
167     print(f"Ridge + взаимодействия: {results[tg]['Ridge + взаимодействия']:.3f}%")
168
169     # 5. Ridge + степени 3-4
170     X_pow = X_int.copy()
171     for fi in imp_idx:
172         X_pow = np.hstack([
173             X_pow,
174             (X_all[:, fi] ** 3).reshape(-1, 1),
175             (X_all[:, fi] ** 4).reshape(-1, 1),
176         ])
177     pred = fit_ridge(X_pow[idx_tr], y_all[tg][idx_tr], X_pow[idx_val])
178     results[tg]["Ridge + степени 3-4"] = mape(y_all[tg][idx_val], pred)
179     print(f"Ridge + степени 3-4: {results[tg]['Ridge + степени 3-4']:.3f}%")
180
181     # Запоминаем лучшую матрицу для этого таргета
182     best_name = min(results[tg], key=results[tg].get)
183     best_matrices[tg] = {
184         "Ridge + квадраты": X_sq,
185         "Ridge + взаимодействия": X_int,
186         "Ridge + степени 3-4": X_pow,
187     }[best_name]
188     best_matrices[tg + "_name"] = best_name
189

```

Рисунок 15. Код построения Ridge-моделей с последовательным расширением признакового пространства.

```

===== target0 =====
OLS:                3.211%
Ridge:              3.211%
Ridge + квадраты:   2.726%
Ridge + взаимодействия: 1.278%
Ridge + степени 3-4: 1.261%

===== target1 =====
OLS:                39.772%
Ridge:              39.770%
Ridge + квадраты:   13.150%
Ridge + взаимодействия: 5.186%
Ridge + степени 3-4: 5.027%

=====
Модель                target0      target1      combined
-----
OLS                    3.211%      39.772%      21.491%
Ridge                  3.211%      39.770%      21.491%
Ridge + квадраты       2.726%      13.150%       7.938%
Ridge + взаимодействия 1.278%       5.186%       3.232%
Ridge + степени 3-4    1.261%       5.027%       3.144%

Лучшая для target0: Ridge + степени 3-4 (1.261%)
Лучшая для target1: Ridge + степени 3-4 (5.027%)

Combined MAPE финального решения: 3.144%
Baseline CatBoost (val):          1.633%

```

Рисунок 16. Результаты обучения всех моделей.

Анализ результатов:

- OLS показала MAPE = 21.491% (среднее). Особенно плохой результат для target1 (39.772%) объясняется существенной нелинейностью зависимости этого таргета от признаков.
- Ridge на исходных признаках дал практически идентичный результат. Это объясняется тем, что RidgeCV подобрал очень маленькое значение λ (близкое к нижней границе) — на 25 признаках переобучения нет, и регуляризация почти не влияет на решение. Эффект от Ridge проявляется только при добавлении большого числа новых признаков, когда растёт мультиколлинеарность.
- Ridge + квадраты дал существенное улучшение для target1 (с 39.77% до 13.15%), что подтверждает наличие выраженной нелинейной зависимости. Для target0 улучшение скромнее (с 3.211% до 2.726%) — вероятно, зависимость target0 от доминирующего признака feature22 более линейна.

- Ridge + взаимодействия снизил ошибку target1 еще более чем в 2.5 раза (с 13.15% до 5.19%), что подтверждает: взаимодействия важных признаков несут существенную информацию о процессе.
- Ridge + степени 3–4 дал небольшое, но стабильное улучшение для обоих таргетов. Финальная модель — Ridge с квадратами, взаимодействиями и степенями 3–4 важных признаков.

Итоговое решение дает Combined MAPE = 3.144%. По сравнению с baseline CatBoost (1.633%) значение выше — и это ожидаемо: линейная модель хуже подгоняется под обучающее распределение, чем градиентный бустинг. Однако ключевое преимущество Ridge заключается не в точности на локальной валидации, а в способности экстраполировать, что и проверяется на следующем шаге.

6. Проверка генерализации

Для проверки способности моделей генерализовать был проведен эксперимент, имитирующий реальную задачу: данные разделены по значению доминирующего признака (для target0 — feature22, для target1 — feature16) на «типичные» (значения в диапазоне 10–90 перцентилей, 80% данных) и «крайние» (значения ниже 10-го или выше 90-го перцентиля, 20% данных). Модели обучались только на типичных значениях, а валидировались на крайних — это воспроизводит ситуацию, когда модель должна работать с данными вне обучающего диапазона.

Процедура была применена как к Ridge (на лучшей матрице признаков), так и к CatBoost — для честного сравнения. Чтобы CatBoost не «подглядывал» в крайние значения при early stopping, «нормальные» данные были дополнительно разделены на внутренние train/val (80/20). Код проверки генерализации приведен на рисунке 17, результаты — на рисунке 18.

```

217 gen_features = {"target0": "feature22", "target1": "feature16"}
218 gen_results = {"Ridge": {}, "CatBoost": {}}
219
220 for tg in TARGETS:
221     fname = gen_features[tg]
222     fi = list(FTS).index(fname)
223     q10, q90 = np.percentile(X_all[:, fi], [10, 90])
224     extreme = (X_all[:, fi] <= q10) | (X_all[:, fi] >= q90)
225     normal = ~extreme
226
227     print(f"\n{tg}: {fname} q10={q10:.2f}, q90={q90:.2f}")
228     print(f"Нормальных: {normal.sum()}, Крайних: {extreme.sum()}")
229
230     # --- Ridge на лучшей матрице ---
231     X_best = best_matrices[tg]
232     pred_ridge = fit_ridge(X_best[normal], y_all[tg][normal], X_best[extreme])
233     gen_results["Ridge"][tg] = mape(y_all[tg][extreme], pred_ridge)
234
235     # --- CatBoost на тех же данных ---
236     # Чтобы не "подглядывать" в extreme для early stopping,
237     # разбиваем normal на внутренние train/val
238     normal_idx = np.where(normal)[0]
239     rng = np.random.RandomState(RS)
240     rng.shuffle(normal_idx)
241     n_inner = int(0.8 * len(normal_idx))
242     inner_tr = normal_idx[:n_inner]
243     inner_val = normal_idx[n_inner:]
244
245     cb = CatBoostRegressor(
246         max_depth=4, iterations=5000,
247         early_stopping_rounds=20, objective="MAPE",
248         verbose=0, random_state=RS
249     )
250     cb.fit(X_all[inner_tr], y_all[tg][inner_tr],
251           eval_set=(X_all[inner_val], y_all[tg][inner_val]))
252     pred_cb = np.clip(cb.predict(X_all[extreme]), 0, 100)
253     gen_results["CatBoost"][tg] = mape(y_all[tg][extreme], pred_cb)
254
255     print(f"    Ridge MAPE:    {gen_results['Ridge'][tg]:.3f}%")
256     print(f"    CatBoost MAPE: {gen_results['CatBoost'][tg]:.3f}%")
257
258 # Итоговая таблица
259 print(f"\n{'=' * 70}")
260 print(f"{'Модель':<15} {'target0':>12} {'target1':>12} {'combined':>12}")
261 print(f"{'-' * 70}")
262 for model in ["Ridge", "CatBoost"]:
263     m0 = gen_results[model]["target0"]
264     m1 = gen_results[model]["target1"]
265     mc = (m0 + m1) / 2
266     print(f"{'model':<15} {m0:>11.3f}% {m1:>11.3f}% {mc:>11.3f}%")
267

```

Рисунок 17. Код проверки генерализации для Ridge и CatBoost.

```

=====
Проверка генерализации: Ridge vs CatBoost
=====

target0: feature22 q10=37.31, q90=122.32
Нормальных: 122733, Крайних: 30684
    Ridge MAPE:    1.016%
    CatBoost MAPE: 2.522%

target1: feature16 q10=42.10, q90=95.33
Нормальных: 122265, Крайних: 31152
    Ridge MAPE:    4.090%
    CatBoost MAPE: 3.995%

=====
Модель                target0      target1      combined
-----
Ridge                  1.016%      4.090%      2.553%
CatBoost               2.522%      3.995%      3.259%

=====
Сравнение: обычная валидация vs крайние значения
=====
Модель                val          extreme      Δ
-----
Ridge + степени 3-4   3.144%      2.553%      -0.591%
CatBoost (baseline)   1.633%      3.259%      +1.626%

```

Рисунок 18. Результаты проверки генерализации.

Анализ результатов:

- Ridge-регрессия при переходе к крайним значениям улучшила MAPE на 0.591% (с 3.144% до 2.553%). Линейная модель с полиномиальными признаками плавно экстраполирует за пределы обучающего диапазона — при отклонении признака от типичных значений предсказание продолжает тренд, а не «упирается» в границу.
- CatBoost при переходе к крайним значениям ухудшил MAPE на 1.626% (с 1.633% до 3.259%). Деревья решений по своей природе не экстраполируют: если значение признака выходит за пределы обучающего диапазона, дерево отвечает так же, как для ближайшей границы. Это классическое поведение и прямое объяснение расхождения локальной валидации (~1.6%) с публичным лидербордом (~10%), которое отмечал автор baseline в README.
- Для target1 CatBoost показал незначительно лучший результат (3.995% против 4.090%), однако для target0 разрыв существенный (2.522% против 1.016% в пользу Ridge). В среднем Ridge-регрессия превосходит baseline на крайних значениях (2.553% против 3.259%), что подтверждает ее преимущество в задаче генерализации.

Вывод

В ходе лабораторной работы было проработано baseline-решение задачи генерализации SIBUR Challenge 2023 на основе CatBoostRegressor и предложено более устойчивое решение на основе Ridge-регрессии с расширением признакового пространства.

1. Воспроизведен baseline организаторов на основе CatBoostRegressor. Локальная валидация показала MAPE = 1.633%, однако по данным автора baseline на публичном лидерборде модели дают ~10% — расхождение в 6 раз указывает на переобучение под train/valid-распределение.

2. Выявлена причина плохой генерализации baseline: деревья решений не экстраполируют — предсказание всегда лежит внутри диапазона обучающей выборки. Для задачи «цифровых экспериментов» это критично.

3. Проверена гипотеза о нелинейности данных. Residual plot линейной модели показал систематическую структуру в остатках (наклонные полосы, выраженные кривые), что подтверждает наличие нелинейных зависимостей. Раскраска остатков по признаку `gas` выявила скрытую категориальную переменную: два режима в данных соответствуют типам газа `gas1` и `gas2`.

4. Проведен анализ важности признаков с помощью CatBoost. Определены доминирующие признаки: `feature22` (62.1%) для target0, `feature16` (49.0%) для target1. Эти признаки использованы для адресного добавления нелинейных преобразований, что позволило избежать взрывного роста числа признаков и сохранить интерпретируемость модели.

5. Построена Ridge-регрессия с последовательным расширением признакового пространства. Показано устойчивое снижение MAPE на каждом шаге: OLS (21.491%) → Ridge (21.491%) → Ridge + квадраты (7.938%) → Ridge + взаимодействия (3.232%) → Ridge + степени 3–4 (3.144%). Финальная модель — Ridge с квадратами, взаимодействиями и степенями 3–4 важных признаков.

6. Проверена генерализация на крайних значениях доминирующих признаков. Ridge показал улучшение MAPE на 0.591% при переходе к крайним значениям (с 3.144% до 2.553%), тогда как CatBoost ухудшился на 1.626% (с 1.633% до 3.259%). Это прямое доказательство того, что линейная модель с полиномиальными признаками решает задачу экстраполяции, которую деревья в принципе не решают.

Итог: несмотря на то что Ridge уступает CatBoost на обычной валидации (3.144% против 1.633%), именно он решает главную задачу хакатона — генерализацию на данных вне обучающей выборки. Цель работы достигнута: изучено baseline-решение, освоены методы L2-регуляризации и feature engineering, предложено решение, превосходящее baseline в той задаче, ради которой проводился хакатон.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

main.py

```
from matplotlib import pyplot as plt

plt.style.use("bmh")
plt.rcParams["axes.titlesize"] = "large"

import pathlib
import numpy as np
import pandas as pd
pd.set_option('display.max_columns', None)

from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_percentage_error
from sklearn.linear_model import LinearRegression, RidgeCV
from sklearn.preprocessing import StandardScaler
from catboost import CatBoostRegressor

# Пути и seed
DATA_DIR = pathlib.Path("./")
RS = 42

# Загрузка данных
train = pd.read_parquet(DATA_DIR.joinpath("train.parquet"))
print("Размер данных:", train.shape)
print(train.head())

# Признаки и таргеты
CAT = ["feature4"]
TARGETS = ["target0", "target1"]
FTS = train.filter(like="feature").columns.difference(CAT)

# Кодирование категориального признака
train["gas"] = 0
train.loc[train.feature4 == "gas2", "gas"] = 1
FTS = FTS.union(["gas"])

# Разделение на train и valid (как в baseline)
X_tr, X_val, y_tr, y_val = train_test_split(
    train[FTS], train[TARGETS], train_size=0.5, random_state=RS
)
print("Train:", X_tr.shape, "Valid:", X_val.shape)

# Residual plot для линейной модели
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for ax, tg in zip(axes, TARGETS):
    lin = LinearRegression().fit(X_tr, y_tr[tg])
    residuals = y_tr[tg] - lin.predict(X_tr)
    for g_val, color in zip([0, 1], ["blue", "orange"]):
        mask = X_tr["gas"] == g_val
        ax.scatter(lin.predict(X_tr)[mask], residuals[mask],
                  alpha=0.1, s=1, c=color, label=f"gas{g_val+1}")
    ax.axhline(0, color="red", linestyle="--")
```

```

ax.legend()
ax.set_xlabel("Предсказание линейной модели")
ax.set_ylabel("Остаток")
ax.set_title(f"Residual plot для {tg}")

plt.tight_layout()
plt.savefig("residuals.png", dpi=100, bbox_inches="tight")
plt.close()

# Обучение CatBoost (baseline организаторов)
models = {}
tr_preds = []
val_preds = []

for tg in TARGETS:
    print(f"{tg}", "=" * 10)
    cb_model = CatBoostRegressor(
        max_depth=4,
        iterations=5000,
        early_stopping_rounds=20,
        objective="MAPE",
        verbose=200,
        random_state=RS
    )
    cb_model.fit(X_tr, y_tr[tg], eval_set=(X_val, y_val[tg]))
    tr_preds.append(cb_model.predict(X_tr))
    val_preds.append(cb_model.predict(X_val))
    models[tg] = cb_model

# Сборка и обрезка
tr_preds = np.clip(np.column_stack(tr_preds), 0, 100)
val_preds = np.clip(np.column_stack(val_preds), 0, 100)

# Метрики
print(f"MAPE (train): {mean_absolute_percentage_error(y_tr, tr_preds) * 100:.3f} %")
print(f"MAPE (val): {mean_absolute_percentage_error(y_val, val_preds) * 100:.3f} %")

# Важность признаков из CatBoost
importances = {}
for tg in TARGETS:
    imp = models[tg].get_feature_importance()
    importances[tg] = imp
    print(f"\n=== Топ-10 признаков для {tg} ===")
    pairs = sorted(zip(FTS, imp), key=lambda x: -x[1])[:10]
    for name, val in pairs:
        print(f"{name}: {val:.2f}")

# Визуализация важности
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
for ax, tg in zip(axes, TARGETS):
    pairs = sorted(zip(FTS, importances[tg]), key=lambda x: x[1])
    names = [p[0] for p in pairs]
    vals = [p[1] for p in pairs]
    ax.barh(names, vals)
    ax.set_title(f"Важность признаков для {tg}")
plt.tight_layout()
plt.savefig("feature_importance.png", dpi=100, bbox_inches="tight")
plt.close()

```

```

# Построение Ridge-регрессии с постепенным расширением признаков
idx_tr, idx_val = train_test_split(
    np.arange(len(train)), train_size=0.5, random_state=RS
)
X_all = train[FTS].values
y_all = {tg: train[tg].values for tg in TARGETS}

def mape(y_true, y_pred):
    return mean_absolute_percentage_error(y_true, y_pred) * 100

def fit_ridge(X_train, y_train, X_valid):
    scaler = StandardScaler()
    Xtr = scaler.fit_transform(X_train)
    Xval = scaler.transform(X_valid)
    model = RidgeCV(alphas=np.logspace(-3, 3, 20), cv=5)
    model.fit(Xtr, y_train)
    return model.predict(Xval)

# Важные признаки, отобранные по результатам CatBoost
important = {
    "target0": ["feature22", "feature2", "feature13", "feature19", "feature20"],
    "target1": ["feature16", "feature13", "feature20", "gas", "feature22"],
}

results = {"target0": {}, "target1": {}}
best_matrices = {} # будем хранить лучшую матрицу признаков для каждого таргета

for tg in TARGETS:
    print(f"\n{'=' * 20} {tg} {'=' * 20}")
    imp_idx = [list(FTS).index(f) for f in important[tg]]

    # 1. OLS
    lin = LinearRegression().fit(X_tr, y_tr[tg])
    results[tg]["OLS"] = mape(y_val[tg], lin.predict(X_val))
    print(f"OLS: {results[tg]['OLS']:.3f}%")

    # 2. Ridge на исходных
    pred = fit_ridge(X_tr, y_tr[tg], X_val)
    results[tg]["Ridge"] = mape(y_val[tg], pred)
    print(f"Ridge: {results[tg]['Ridge']:.3f}%")

    # 3. Ridge + квадраты
    X_sq = np.hstack([X_all, X_all ** 2])
    pred = fit_ridge(X_sq[idx_tr], y_all[tg][idx_tr], X_sq[idx_val])
    results[tg]["Ridge + квадраты"] = mape(y_all[tg][idx_val], pred)
    print(f"Ridge + квадраты: {results[tg]['Ridge + квадраты']:.3f}%")

    # 4. Ridge + взаимодействия
    X_int = X_sq.copy()
    for fi in imp_idx:
        for j in range(X_all.shape[1]):
            if j != fi:
                X_int = np.hstack([X_int, (X_all[:, fi] * X_all[:, j]).reshape(-1, 1)])
    pred = fit_ridge(X_int[idx_tr], y_all[tg][idx_tr], X_int[idx_val])
    results[tg]["Ridge + взаимодействия"] = mape(y_all[tg][idx_val], pred)
    print(f"Ridge + взаимодействия: {results[tg]['Ridge + взаимодействия']:.3f}%")

    # 5. Ridge + степени 3-4
    X_pow = X_int.copy()

```

```

for fi in imp_idx:
    X_pow = np.hstack([
        X_pow,
        (X_all[:, fi] ** 3).reshape(-1, 1),
        (X_all[:, fi] ** 4).reshape(-1, 1),
    ])
    pred = fit_ridge(X_pow[idx_tr], y_all[tg][idx_tr], X_pow[idx_val])
    results[tg]["Ridge + степени 3-4"] = mape(y_all[tg][idx_val], pred)
    print(f"Ridge + степени 3-4: {results[tg]['Ridge + степени 3-4']:.3f}%")

# Запоминаем лучшую матрицу для этого таргета
best_name = min(results[tg], key=results[tg].get)
best_matrices[tg] = {
    "Ridge + квадраты": X_sq,
    "Ridge + взаимодействия": X_int,
    "Ridge + степени 3-4": X_pow,
}[best_name]
best_matrices[tg + "_name"] = best_name

# Итоговая таблица
print(f"\n{'=' * 60}")
print(f"{'Модель':<30} {'target0':>12} {'target1':>12} {'combined':>12}")
print(f"{'-' * 60}")
for model in results["target0"].keys():
    m0 = results["target0"][model]
    m1 = results["target1"][model]
    mc = (m0 + m1) / 2
    print(f"{'model':<30} {'m0':>11.3f}% {'m1':>11.3f}% {'mc':>11.3f}%")

# Финальные модели
best0 = min(results["target0"], key=results["target0"].get)
best1 = min(results["target1"], key=results["target1"].get)
mape_best0 = results["target0"][best0]
mape_best1 = results["target1"][best1]
mape_combined = (mape_best0 + mape_best1) / 2

print(f"\nЛучшая для target0: {best0} ({mape_best0:.3f}%)")
print(f"Лучшая для target1: {best1} ({mape_best1:.3f}%)")
print(f"\nCombined MAPE финального решения: {mape_combined:.3f}%")
print(f"Baseline CatBoost (val): 1.633%")

# Проверка генерализации: Ridge vs CatBoost
print(f"\n{'=' * 70}")
print("Проверка генерализации: Ridge vs CatBoost")
print(f"{'-' * 70}")

gen_features = {"target0": "feature22", "target1": "feature16"}
gen_results = {"Ridge": {}, "CatBoost": {}}

for tg in TARGETS:
    fname = gen_features[tg]
    fi = list(FTS).index(fname)
    q10, q90 = np.percentile(X_all[:, fi], [10, 90])
    extreme = (X_all[:, fi] <= q10) | (X_all[:, fi] >= q90)
    normal = ~extreme

    print(f"\n{tg}: {fname} q10={q10:.2f}, q90={q90:.2f}")
    print(f"Нормальных: {normal.sum()}, Крайних: {extreme.sum()}")

```

```

# --- Ridge на лучшей матрице ---
X_best = best_matrices[tg]
pred_ridge = fit_ridge(X_best[normal], y_all[tg][normal], X_best[extreme])
gen_results["Ridge"][tg] = mape(y_all[tg][extreme], pred_ridge)

# --- CatBoost на тех же данных ---
# Чтобы не "подглядывать" в extreme для early stopping,
# разбиваем normal на внутренние train/val
normal_idx = np.where(normal)[0]
rng = np.random.RandomState(RS)
rng.shuffle(normal_idx)
n_inner = int(0.8 * len(normal_idx))
inner_tr = normal_idx[:n_inner]
inner_val = normal_idx[n_inner:]

cb = CatBoostRegressor(
    max_depth=4, iterations=5000,
    early_stopping_rounds=20, objective="MAPE",
    verbose=0, random_state=RS
)
cb.fit(X_all[inner_tr], y_all[tg][inner_tr],
      eval_set=(X_all[inner_val], y_all[tg][inner_val]))
pred_cb = np.clip(cb.predict(X_all[extreme]), 0, 100)
gen_results["CatBoost"][tg] = mape(y_all[tg][extreme], pred_cb)

print(f" Ridge MAPE: {gen_results['Ridge'][tg]:.3f}%")
print(f" CatBoost MAPE: {gen_results['CatBoost'][tg]:.3f}%")

# Итоговая таблица
print(f"\n{'=' * 70}")
print(f"{'Модель':<15} {'target0':>12} {'target1':>12} {'combined':>12}")
print("-" * 70)
for model in ["Ridge", "CatBoost"]:
    m0 = gen_results[model]["target0"]
    m1 = gen_results[model]["target1"]
    mc = (m0 + m1) / 2
    print(f"{'model':<15} {'m0':>11.3f}% {'m1':>11.3f}% {'mc':>11.3f}%")

# Итоговое сравнение с обычной валидацией
print(f"\n{'=' * 70}")
print("Сравнение: обычная валидация vs крайние значения")
print("-" * 70)
print(f"{'Модель':<25} {'val':>10} {'extreme':>12} {'Δ':>10}")
print("-" * 70)
# Ridge
ridge_val = mape_combined
ridge_ext = (gen_results["Ridge"]["target0"] + gen_results["Ridge"]["target1"]) / 2
print(f"{'Ridge + степени 3-4':<25} {'ridge_val':>9.3f}% {'ridge_ext':>11.3f}% {'ridge_ext - ridge_val':>+9.3f}%")
# CatBoost
cb_ext = (gen_results["CatBoost"]["target0"] + gen_results["CatBoost"]["target1"]) / 2
print(f"{'CatBoost (baseline)':<25} {1.633:>9.3f}% {'cb_ext':>11.3f}% {'cb_ext - 1.633':>+9.3f}%")_

```